

Newton's Method for Nonlinear Equations

Geometric idea, convergence, and a physics application

Sebastián Vargas Contreras

14/09/2026

Outline

- 1 The Problem
- 2 Geometric Idea: Why the Tangent?
- 3 Derivation and Algorithm
- 4 Worked Example
- 5 Physical Application: Free Fall with Air Resistance
- 6 Summary

Solving $f(x) = 0$

Many problems in science and engineering reduce to finding a value x^* such that

$$f(x^*) = 0,$$

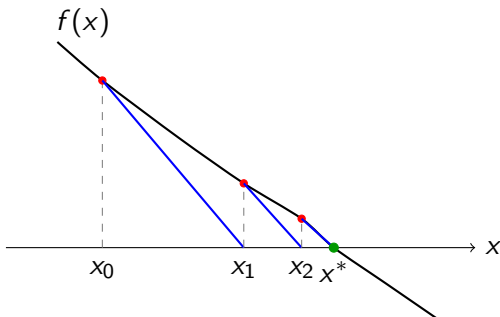
where f is, in general, a **nonlinear** function.

Examples:

- Roots of polynomials.
- Equilibrium points of physical systems.
- Orbital mechanics (Kepler's equation).
- Optimization problems (finding critical points of f').

Since most nonlinear equations cannot be solved algebraically, we need **iterative numerical methods**. Newton's method is the most widely used one.

Why the tangent line?



- Near x_n , the tangent line is the **best local linear approximation** of f .
- Finding the root of a *straight line* is trivial no curve to solve.
- If x_n is close enough to the root, the tangent's root x_{n+1} is usually even closer.
- Repeating this process drives $x_n \rightarrow x^*$ very quickly.

From geometry to a formula

The tangent line to f at the point $(x_n, f(x_n))$ is

$$y = f(x_n) + f'(x_n)(x - x_n).$$

We look for the point where this line crosses the x -axis ($y = 0$) and call it x_{n+1} :

$$0 = f(x_n) + f'(x_n)(x_{n+1} - x_n)$$

Solving for x_{n+1} gives **Newton's iteration formula**:

$$x_{n+1} = x_n - \frac{f(x_n)}{f'(x_n)}$$

The algorithm

- 1 Choose an initial guess x_0 (as close as possible to the root).
- 2 Evaluate $f(x_n)$ and $f'(x_n)$.
- 3 Compute $x_{n+1} = x_n - f(x_n)/f'(x_n)$.
- 4 Check a stopping criterion: $|x_{n+1} - x_n| < \varepsilon$ or $|f(x_{n+1})| < \varepsilon$.
- 5 If not satisfied, set $n = n + 1$ and repeat from step 2.

- **Quadratic convergence:** near a simple root ($f'(x^*) \neq 0$), the number of correct digits roughly *doubles* at every step.
- Requires f to be differentiable and $f'(x_n) \neq 0$.
- Sensitive to the initial guess x_0 : a poor choice can cause divergence, oscillation, or convergence to the wrong root.
- For a root of multiplicity > 1 , convergence degrades to only linear.

Example: computing $\sqrt{2}$

Solve $f(x) = x^2 - 2 = 0$, so $f'(x) = 2x$ and

$$x_{n+1} = x_n - \frac{x_n^2 - 2}{2x_n}.$$

n	x_n	$f(x_n)$
0	1.000000	-1.000000
1	1.500000	0.250000
2	1.416667	0.006944
3	1.414216	0.000006

After only 3 iterations we already have 6 correct decimal digits of $\sqrt{2} \approx 1.414214$ a clear illustration of quadratic convergence.

Physical application: falling with air resistance

An object of mass m falls from rest under gravity g , with linear air drag proportional to its velocity. Its position after falling for a time t is

$$s(t) = v_t t - v_t \tau \left(1 - e^{-t/\tau}\right),$$

where v_t is the **terminal velocity** and $\tau = v_t/g$ is a time constant.

Question: how long does it take to fall a height H ? We need to solve $s(t) = H$ for t but t appears both linearly and inside an exponential, so the equation is **transcendental**.

Setting up Newton's method

Define

$$f(t) = s(t) - H = v_t t - v_t \tau (1 - e^{-t/\tau}) - H.$$

A nice feature of this problem: the derivative of f is simply the object's **instantaneous velocity**,

$$f'(t) = v_t (1 - e^{-t/\tau}) = v(t).$$

The iteration becomes

$$t_{n+1} = t_n - \frac{f(t_n)}{v(t_n)}.$$

A convenient initial guess ignores air resistance: $t_0 = H/v_t$.

Python implementation

```
import numpy as np

def fall_time(H, v_t, g=9.8, tol=1e-8, max_iter=100):
    """Time to fall a height H with linear air drag (Newton's method)."""
    tau = v_t / g
    t = H / v_t # naive initial guess (no air resistance)
    for i in range(max_iter):
        f = v_t*t - v_t*tau*(1 - np.exp(-t/tau)) - H
        v = v_t*(1 - np.exp(-t/tau)) #  $f'(t)$  = instantaneous velocity
        t_new = t - f / v
        if abs(t_new - t) < tol:
            return t_new, i + 1
        t = t_new
    raise RuntimeError("Newton's method did not converge")

# Skydiver: terminal velocity 55 m/s, falls H = 1000 m
v_t, H = 55.0, 1000.0
t, n_iter = fall_time(H, v_t)

print(f"Fall time t = {t:.6f} s")
print(f"Converged in {n_iter} iterations")
print(f"Impact velocity = {v_t*(1 - np.exp(-t*9.8/v_t)):.4f} m/s")
```

Results

For $v_t = 55$ m/s and $H = 1000$ m, starting from the naive guess $t_0 = H/v_t \approx 18.18$ s:

n	t_n (s)	$f(t_n)$
0	18.181818	-296.580617
1	23.794063	4.448712
2	23.711995	0.000478
3	23.711986	0.000000

Newton's method converges to $t \approx 23.712$ s in only **4 iterations** much faster than bisection, and far easier than isolating t from the exponential by hand.

Summary

- Newton's method replaces the curve f by its tangent line at each step and uses the tangent's root as the next approximation.
- Formula: $x_{n+1} = x_n - f(x_n)/f'(x_n)$.
- Converges quadratically near a simple root very fast once close enough.
- Needs f' and a good initial guess; not guaranteed to converge globally.
- Widely used in physics and engineering, e.g. finding the fall time of an object with air resistance.
- When f' is unavailable or unreliable, the **secant method** (which approximates f' with finite differences) is a common alternative.

Thank you!

Questions?